

选题	2024 年第十四届 APMCM 亚太地区大学生数学建模竞赛（中文赛项）	参赛编号
C		apmcm 24100023

基于 QUBO 模型的物流配送优化

摘要

本文研究了基于量子计算的物流配送问题，旨在通过量子计算技术优化物流配送方案，降低成本并提高效率。首先，本文介绍了量子计算方法的高效性，尤其是相干伊辛机（CIM）在解决复杂优化问题方面的优势。接着，本文以物流配送问题的数学模型为基础构建了 QUBO 模型，并利用 Kaiwu SDK 中的 CIM 模拟器和模拟退火求解器进行求解。最后，本文将 QUBO 模型的应用拓展到金融领域。

针对问题一：对于单个物流公司运营成本最小化问题，本文首先通过迪杰斯特拉算法求出最短路径，而后建立了一般数学模型并转化为 QUBO 模型，然后通过 CIM 模拟器和模拟退火求解器进行求解，得到了最佳货车租赁方案和货物运输方案，并计算出公司一和公司二的最低总成本分别为 197500 元和 172000 元。此外，本文还通过与贪心算法和遗传算法的对比，展现了 QUBO 模型的优势，包括计算速度快、精度高等。

针对问题二：在允许两公司之间合作拼货运输的条件下，本文在第一问的基础上更新了一般数学模型并转化为 QUBO 模型，同时分解为若干个 SubQUBO 模型利用 CIM 模拟器和模拟退火求解器求解，并用贪心算法和遗传算法加以比较验证，得到了最佳货车租赁方案和货物运输方案，并计算出两公司合作时的最低总成本为 318000 元，比分别单独运输时节约了 51500 元，由此证明了合作拼货运输可以降低两公司总成本，提高物流效率。

针对问题三：本文提出了一个金融领域基于 QUBO 模型的信用评分卡优化场景，并建立了对应的 QUBO 模型表达式。通过分析模型涉及的变量数量，计算得出模型所需的比特数量级，为后续研究和应用提供了参考。

本文将量子计算技术应用于物流配送优化问题，通过 QUBO 模型实现了最小化运营总成本的目标，并分析了合作拼货运输的优势，同时还利用贪心算法和遗传算法验证了 QUBO 模型的优越性。此外，本文还提出了一个具有学术价值和应用前景的金融领域的信用评分卡优化场景，为量子计算技术在更多领域的应用提供了思路。

关键词：物流配送优化、QUBO 模型、CIM、模拟退火、贪心算法、遗传算法

目 录

一、问题重述与背景介绍	1
1.1 问题重述	1
1.2 背景介绍	1
1.2.1 二次无约束二进制优化(QUBO)模型:	1
1.2.2 CIM 模拟器	2
1.2.3 模拟退火算法	2
二、问题分析	4
2.1 问题总分析	4
2.2 问题一的分析	4
2.3 问题二的分析	5
2.4 问题三的分析	5
三、模型假设	5
四、符号说明	6
五、模型建立与求解	6
5.1 问题一模型的建立与求解	6
5.1.1 数据预处理	6
5.1.2 构建数学模型和 QUBO 模型	8
5.1.3 模型的求解	11
5.1.4 问题一模型检验	13
5.2 问题二模型的建立与求解	13
5.2.1 问题二模型建立	13
5.2.2 问题二模型求解	16
5.3 问题三模型	17
5.3.1 问题背景及实际意义	17
5.3.2 符号设计	18
5.3.3 模型建立与求解	18
六、模型评价、改进与推广	19
6.1 模型的优点	19
6.2 模型的缺点	20
6.3 模型的改进与推广	20
七、参考文献	21
八、附录	22

一、问题重述与背景介绍

1.1 问题重述

电子商务的飞速发展使得物流配送需求日益增长。面对复杂的运输需求，传统的物流优化方法已不足以高效地解决物流问题，故而催生了新型的量子计算技术来自动计算运输综合策略，物流公司得以有更高效的方法来更有效地进行物流配送安排，实现物流效率的提高、运输成本的降低及消费者满意度的提升。

相干伊辛机在处理相关复杂优化问题中表现出巨大的潜力，故而本问题要求在物流配送场景下，将问题建立在与相干伊辛机密切相关的 QUBO 模型上，使用 Kaiwu SDK 进行求解。

问题一：在拼货只分别发生在各物流公司内部的前提下，以最小化单个物流公司的运营成本为目标，在总要求的方法下，使用其中的 CIM 模拟器和模拟退火求解器分别设计出两个物流公司的最佳货车租赁方案和货物运输方案。

问题二：在两公司之间可以合作拼货运输的前提下，以最小化两公司总成本为目标，用 Kaiwu SDK 中的 CIM 模拟器和模拟退火求解器得出最佳货车租赁方案和货物运输方案，并计算其相较于问题一中得到的方案对于两公司总成本的减少量。

问题三：提出一个具有学术价值或商业化前景的设想，给出其对应的 QUBO 模型表达式，并计算模型所需比特数量级。

1.2 背景介绍

1.2.1 二次无约束二进制优化(QUBO)模型：

1.QUBO 模型的定义：

QUBO 模型是指二次无约束二值优化模型，它是一种用于解决组合优化问题的数学模型。在 QUBO 模型中，需要将问题转化为一个决策变量为二值变量，目标函数是一个二次函数形式优化模型，一般定义为下式：

$$y = x^T Q x$$

其中， Q 为 QUBO 矩阵， x 为二进制变量组成的向量，即 x 为 0-1 变量，其取值为 $\{0,1\}$ ，QUBO 目标为找到使得 y 达到最值的 x 。

Q 矩阵的形式有以下 2 种：

①对称形式

$$q_{ij}' = \frac{q_{ij} + q_{ji}}{2}$$

②上三角形形式

$$\begin{aligned} q_{ij}' &= (q_{ij} + q_{ji}), i \leq j \\ q_{ij}' &= 0, i > j \end{aligned}$$

当添加约束条件后转化为以下形式：

$$\begin{aligned} \min f(x) \\ s.t. Ax = b \\ f'(x) = f(x) + P(Ax - b)^T(Ax - b) \end{aligned}$$

其中， P 表示正标量惩罚项。随着问题规模增加，若仍使用传统算法求解组合优化问题，将会大大增加求解的时间，如若利用 QUBO 模型，即可运行在量子计算机硬件上，通过量子计算机进行毫秒级别的加速求解，得以更高效地求解组合优化问题。

2. QUBO 模型中约束条件的转化：

QUBO 模型为二次无约束优化问题，故如要实际解决一些有约束条件的问题，需要在目标函数中引入二次惩罚项，从而将组合优化问题重新进行表述为 QUBO 模型。同类型的约束关系可以用惩罚函数以非常自然的方式体现在“无约束”QUBO 公式中^[1]。在 QUBO 公式中，使用惩罚函数可以产生比较精确的模型表示，一些经典约束的惩罚项转换实现如下：

经典约束	等效惩罚
$x + y \leq l$	$P(x y)$
$x + y \geq l$	$P(l - x - y + x y)$
$x + y = l$	$P(l - x - y + 2 x y)$
$x \leq y$	$P(x - x y)$
$x_1 + x_2 + x_3 \leq l$	$P(x_1 x_2 + x_1 x_3 + x_2 x_3)$
$x = y$	$P(x + y - 2 x y)$

其中，函数 P 为正的足够大的标量惩罚值。惩罚项指定的规则是：对于最小化问题的求解，如果是可行解，即满足约束条件，对应的惩罚项等于零；对于不可行解，即不满足约束条件的解，其惩罚项等于一些正的惩罚量。 P 值的选取是该惩罚项设置好坏的关键因素。

1.2.2 CIM 模拟器

相干伊辛机 (Coherent Ising Machine, 简称 CIM)，是一种基于光学技术的量子优化装置，主要用于解决复杂优化问题。相干伊辛机与 QUBO 模型联系紧密，将优化问题转化为 QUBO 模型后，可运用基于相干伊辛机求解 QUBO 模型的软件开发套件——Kaiwu SDK 进行求解，具体操作 CIM 模拟器的流程如下：

步骤一：在 Python 3.8 环境下安装并导入 Kaiwu SDK 库。

步骤二：输入本文中经转化而得的 QUBO 模型。

步骤三：选择 CIM 模拟求解器，模拟在相干光量子计算机上求解 QUBO 模型。

步骤四：设置 CIM 模拟器的参数。

步骤五：调用 Kaiwu SDK 库中提供的方法，将构建好的 QUBO 模型传递给 CIM 模拟器进行求解。

步骤六：获取结果。由于变量为二进制形式，故求得结果为二进制形式。

步骤七：将结果转化表示为实际问题所需结果。

1.2.3 模拟退火算法

模拟退火算法 (Simulated Annealing, SA) 是一种通用的全局优化算法，用于在搜索空间中找到全局最优解。它的基本原理是模拟物理学中的退火过程，从某一较高初始温度出发，通过温度参数控制搜索过程中接受次优解的概率，以一定概率接受比当前解

更差的解，即在局部最优解能概率性地跳出并最终趋于全局最优，从而避免陷入局部最优^[2]。模拟退火算法的具体思路如图 4 所示：

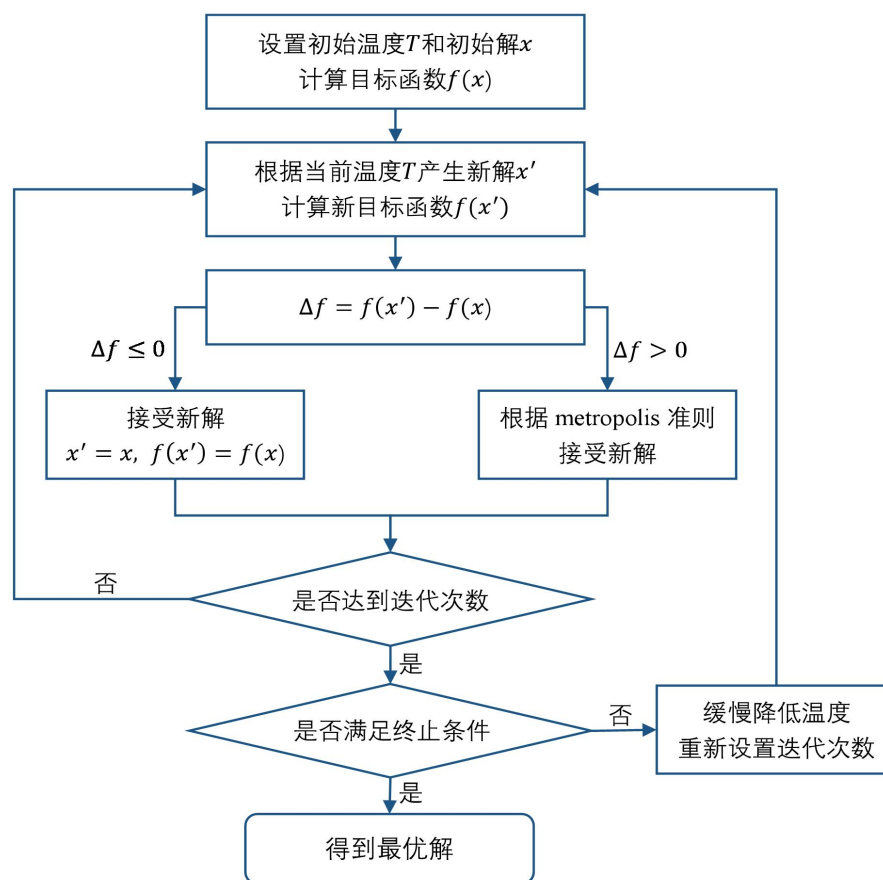


图 4 模拟退火算法流程图

关于流程图中部分内容的说明：

(1) Metropolis 准则

Metropolis 准则是一种有效的重点抽样法。其思想为：系统从一个当前能量状态变化到新的能量状态。若新的能量状态小于当前能量状态，则以概率 1 接受新的能量状态（取得局部最优）；若新的能量状态大于当前能量状态（全局搜索最优点），则以一定的概率接受或舍弃新的能力状态^[3]。其数学性表示为：

$$p = \begin{cases} 1 & , \text{ if } E_{T_{new}} < E_{T_{cur}} \\ \exp\left(-\frac{E_{T_{new}} - E_{T_{cur}}}{T}\right) & , \text{ if } E_{T_{new}} > E_{T_{cur}} \end{cases}$$

（其中 $E_{T_{new}}$ 表示当前温度 T 下新的状态能量， $E_{T_{cur}}$ 表示当前温度 T 下当前状态能量）

从方程可得，当新的能量状态大于当前状态能量时，温度越高，新状态的接受概率越高；当前状态的能量与新状态的能量差越高，新状态的接受概率越高。为避免所谓热力学中淬炼（quenching）的操作，控制参数 T 的值必须缓慢衰减。

(2) 几个重要参数的选择

① 初始控制参数温度 T ：

一般情况下采用随机生成一组可行解，以该解所对应的目标函数值的方差作为初温；利用经验或试验确定。

② 退温函数：

常用的退温函数有以下两种：

$$T_{n+1} = kT_n$$
$$T_{n+1} = T_n - \Delta T$$

其中 n 为当前循环次数， $0 < k < 1$ 为一个非常接近 1 的常数， ΔT 为自定义温度下降值。

③ 内循环的循环次数：

每一个温度下的迭代次数相同，且迭代次数与具体问题有关。随着温度的下降，迭代次数应增加。

④ 算法终止准则：

主要包括以下三点：设置终止温度；设置最大迭代次数；算法搜索到的最优值连续若干步保持不变。

二、问题分析

2.1 问题总分析

本题为基于量子计算的物流配送问题，问题一及问题二均要求设计出最佳的物流配送方案，属于最优化问题。其中引入了量子计算技术以更好地进行物流方案设计，通过建立量子计算中的 QUBO 模型，而后代入基于相干伊辛机开发的 Kaiwu SDK 开发套件进行求解，得到最优物流方案，问题的关键在于将问题的公式转化为 QUBO 形式，并调用 Kaiwu 库里的 CIM 模拟器和模拟退火求解器进行求解。问题的基本思路如图 1 所示：

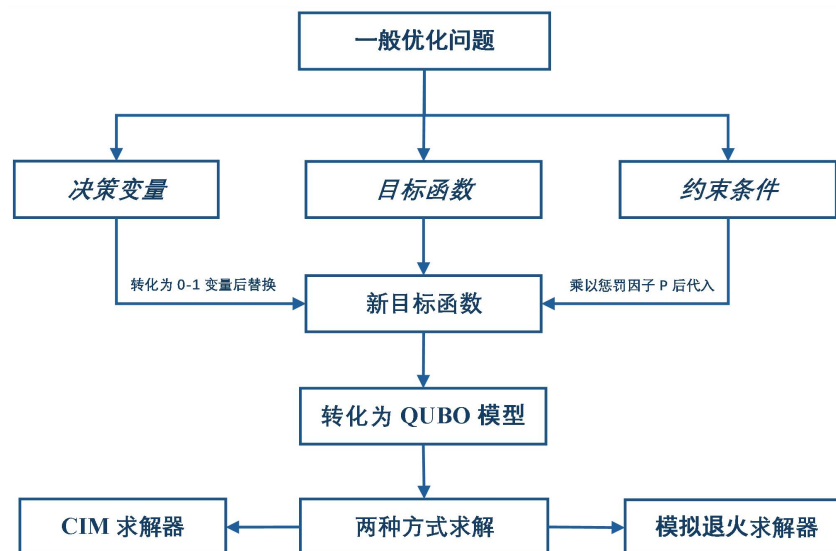


图 1 总体思路流程图

2.2 问题一的分析

针对问题一，判断其为优化问题。首先，通过观察与计算可得，存在两地之间直接运输的单趟成本高于经过另一地中转的单趟运输成本的情况，故而将各城市及其之间的

运输成本抽象为图中的点与权重，运用 Dijkstra 算法得到任意两点之间的最短路径和最短路径长度，从而得到卡车从城市 i 到城市 j 的运输总成本的最小值。而后通过对问题的分析，得到卡车 A、B 的最大租赁数量，将模型简化。接着，以最小化单个公司的运输成本为目标，将约束条件：①卡车载货量的重量限制；②存运货数量的等量关系；③对变量实际意义的约束转化为数学表达式，将其表示为一般优化模型，再通过对对应关系，将其转化为 QUBO 模型，而后通过 python 程序编程，调用 Kaiwu SDK 库中的 CIM 模拟器和模拟退火求解器分别求解，得到最佳货车租赁方案、货物运输方案及航空运输的安排。再通过与一般优化模型中的遗传算法与贪心算法得到的结果及运算时间进行比较，证明本题所用的 QUBO 模型的先进性。

2.3 问题二的分析

针对问题二，判断其为与问题一类似的优化问题。基于问题一中对于最短路径的处理，增加公司一、二可在途中进行拼货运输的条件，则在运输过程中任一卡车上的货物可能含公司一或公司二的，也可能同时包含两公司的货物，即用 $q_{ij,n}^{K,M}$ 表示公司 M 从城市 i 到城市 j 通过卡车 K 所运载的货物数量，在运输过程中， M 可取 1 和 2。由于公司间拼货条件的增加，对于卡车 A 最大租赁数量的限制有所改变，进行新的数量分析，简化模型。本题的目标为最小化两公司总成本，约束条件除改变中途两公司间可以拼货的情况其余与问题一中的约束相同。而后将目标函数与约束条件转化为 QUBO 模型，判断出该模型的比特数较高，使用 SubQUBO 进行求解，调用 Kaiwu SDK 中的 CIM 模拟器和模拟退火求解器进行求解，得到最优安排。再分别计算问题一及问题二中两公司的运输成本总和，计算得出两公司合作下总体成本的减少量。

2.4 问题三的分析

针对问题三，其要求提出具有学术价值及商业化前景的场景。量子计算的优势之一是可以处理大规模的数据，考虑到此优势，通过对附件论文的研究及对资料的查询，本文将视野放至金融领域，对于银行在借款中所会面对的用户正常交纳利息及坏账的可能，选择对用户最佳的评价标准决定是否向该用户放贷，从而使银行获得最大利润。基于此问题背景列出公司收益的表达式，并将其转化为 QUBO 模型，并计算该模型所需的比特数量级。

三、模型假设

1. 假设每个城市有足够的可租赁卡车；
2. 假设货物可以暂存在城市中，由后续车辆将其运走；
3. 假设航空运输无论远近每吨运费皆相同，且能当日到达；
4. 假设问题一种公司一的货物运往小组一的城市，公司二的货物运往小组二的城市；
5. 假设运输天数与题目所给天数严格相等，不考虑交通、天气等不确定因素的影响；
6. 假设不考虑两公司相同城市的货仓间的距离；
7. 假设天数不足一天时均按照一天计算。

四、符号说明

符号	说明	单位
a_{ij}	卡车 A 从城市 <i>i</i> 到城市 <i>j</i> 的运输总成本	元
b_{ij}	卡车 B 从城市 <i>i</i> 到城市 <i>j</i> 的运输总成本	元
t_{ij}	卡车从城市 <i>i</i> 到城市 <i>j</i> 所需的运输天数	天
d_K	卡车 K 的日租金	元
e_{ij}	卡车从城市 <i>i</i> 到城市 <i>j</i> 的运输单趟成本	元
$x_{ij,n}^K$	第 <i>n</i> 辆 <i>K</i> 类型卡车是否参与城市 <i>i</i> 到城市 <i>j</i> 的运输($i, j = 1, \dots, 6$)	/
$q_{ij,n}^{K,M}$	公司 M 卡车 $x_{ij,n}^K$ 上运载的货物量($i, j = 1, \dots, 6$)	吨
$c_{ij,M}$	公司 M 的航空运输量(此处 $i = 1, 2, 6; j = 3, 4, 5$)	吨
C	航空运输每吨的成本(为常量 10000)	元/吨
T_{Mi}	公司 M 在出发城市 <i>i</i> 的初始货物量(此处 $i = 1, 2, 6$)	吨
D_{Mj}	公司 M 需运往到达城市 <i>j</i> 的货物量(此处 $j = 3, 4, 5$)	吨

五、模型建立与求解

5.1 问题一模型的建立与求解

5.1.1 数据预处理

由题中所给卡车日租金、卡车单趟时间及单趟成本表可计算得到 A、B 两种卡车在不同地之间运送货物的总成本，以 A 卡车从*i*地运输货物到*j*地为例，即：

$$a_{ij} = t_{ij}d_A + e_{ij}$$

运用 Excel 公式编写，计算得到两地点间卡车直达运输的总成本，结果如表格 1 所示：

表 1 卡车 A 在各地间的直达运输总成本

	上海	西安	昆明	深圳	天津	郑州
上海	0	28500	56500	36500	28500	18500
西安	28500	0	38500	40500	20500	8500
昆明	56500	38500	0	17500	61500	49500
深圳	36500	40500	17500	0	50500	37500
天津	28500	20500	61500	50500	0	10500
郑州	18500	8500	49500	37500	10500	0

通过对表格中数据的观察和计算可得，将*i*地货物运送到*j*地的过程中，存在比两地间直接运输更节约总成本的中转运输方式，即存在中转地*v*，使得：

$$a_{iv} + a_{vj} < a_{ij}$$

故而本文运用 Dijkstra 算法，不断更新最短路径和路径长度，更新得：

$$a_{ij} = a_{iv} + a_{vj}$$

最终得到任意两点之间的最短路径及路径长度 a_{ij} ，具体实现见附录一。为简化模型，此处地点 i 运输货物到地点 j 不考虑在中转地点 v 处的拼货情况。更新后的最短路径图如图 2、3 所示，最短路径矩阵如图 4 所示，最短路径长度即两地间最低运输总成本导出至 Excel 表后的结果如表 2、3 所示：

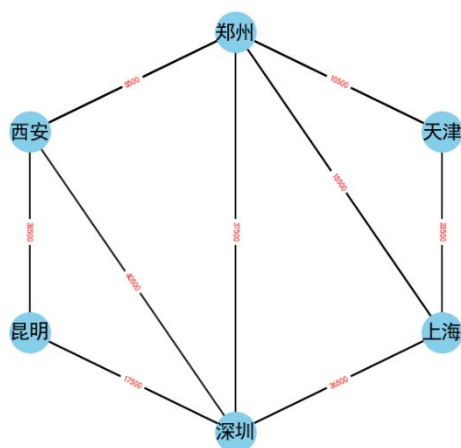


图 2 卡车 A 总费用最短路径图

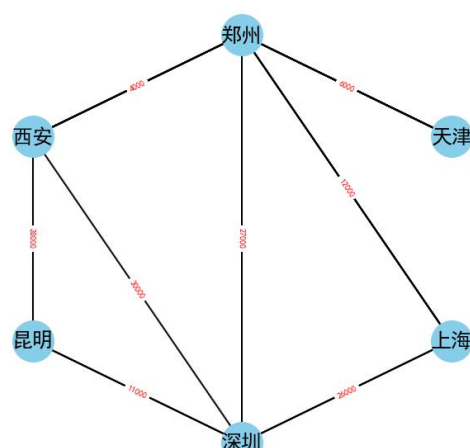


图 3 卡车 B 总费用最短路径图

$$\begin{pmatrix} 0 & 6 & 4 & 0 & 0 & 0 \\ 6 & 0 & 0 & 0 & 6 & 0 \\ 4 & 0 & 0 & 0 & 6 & 2 \\ 0 & 0 & 0 & 0 & 6 & 0 \\ 0 & 6 & 6 & 6 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 & 0 \end{pmatrix}$$

图 4 最短路径矩阵

表 2 卡车 A 在各地间的最低运输总成本

	上海	西安	昆明	深圳	天津	郑州
上海	0	28500	54000	36500	28500	18500
西安	28500	0	38500	40500	19000	8500
昆明	54000	38500	0	17500	61500	47000
深圳	36500	40500	17500	0	50500	37500
天津	28500	19000	61500	50500	0	10500
郑州	18500	8500	47000	37500	10500	0

表 3 卡车 B 在各地间的最低运输总成本

	上海	西安	昆明	深圳	天津	郑州
上海	0	16000	37000	26000	18000	12000
西安	16000	0	28000	30000	10000	4000
昆明	37000	28000	0	11000	38000	32000
深圳	26000	30000	11000	0	33000	27000
天津	18000	10000	38000	33000	0	6000
郑州	12000	4000	32000	27000	6000	0

而后为了衡量最短路径的准确性，本文运用 Floyd 算法进行检验，通过比较得到两种算法的结果相同，故而认为所得结果即为最短路径。Floyd 算法的具体实现见附录 2。

5.1.2 构建数学模型和 QUBO 模型

一、模型抽象及简化

1. 将城市抽象为字母

依题意，城市域 $J = \{\text{上海, 西安, 昆明, 深圳, 天津, 郑州}\}$ ，按顺序分别用数字 1-6 表示。其中上海、西安、郑州为货物当前所在城市，即出发城市，用 T_{Mi} 表示公司 M 在出发城市 i 的初始货物量，故此处 $i = 1, 2, 6$ ；昆明、深圳、天津为货物需要运往的城市，即到达城市，用 D_{Mj} 表示公司 M 需运往到达城市 j 的货物量，故此处 $j = 3, 4, 5$ 。由于拼货情况的存在，故每一个城市都可能作为发货点及收货点，设 $x_{ij,n}^K$ 表示第 n 辆 K 类型卡车对于城市 i 到城市 j 的运输的参与情况，若参与运输则取 1，若无参与则取 0，并用 $q_{ij,n}^{K,M}$ 表示公司 M 中卡车 $x_{ij,n}^K$ 上运载的货物量，此时 $i, j = 1, 2, \dots, 6$ 。

2. 对 $x_{ij,n}^K$ 进行分析

(1) $x_{ij,n}^K$ 表示货车 K 经最优路线由 i 城市发出到达 j 城市，使得运输成本达到最低，由最短路径矩阵可知，该路线可能会途径其他城市 v ，但该模型认为 K 货车在此运输过程不在 v 城市做停留。

(2) 对于 n :

① $K = A$ 时

由于到达城市中对于货量的最大需求为 27 吨，卡车 A 的最大载重量为 12 吨，派用两辆卡车 A 后还剩 $27 - 2 \times 12 = 3$ (吨) < 5 (吨) 的货物没有运输，此为用两辆卡车 A 后所剩货物量最多的情况，而此最大情况所剩的货物量都较小，此时使用卡车 B 或航空运输运送剩余部分的成本相较于使用卡车 A 的成本更低，故使用卡车 A 的最大数量为 2，即此时 $n = 1, 2$ 。

② $K = B$ 时

由于两辆卡车 B 的最大运输量为 10 吨，小于一辆卡车 A 的最大运输量 12 吨，但两辆卡车 B 的租金大于一辆卡车 A 的租金，又因单趟运输成本相同，故有 $2b_{ij} < a_{ij}$ ，故租两辆卡车 B 的方案差于租一辆卡车 A 的方案，故不考虑租两辆卡车 B 及以上的情况，故所租卡车 B 的最大数量为 1，即此时 $n = 1$ 。

二、构建优化模型

由于本题要求分别求得公司一、二的最优运输安排，故以下仅表示出公司一的最优运输安排的求解过程，公司二的情况仅改变出发城市与最终收货城市的货量，其余处理方式相同。

(一) 决策变量

1. 卡车运输的决策变量

用 $x_{ij,n}^K$ 表示第 n 辆卡车 K 是否参与城市 i 到城市 j 的运输，是则取 1，不是则取 0。

则 $x_{ij,n}^K$ ，即卡车 A 、 B 的运输安排共同影响最终的运输成本。

2. 航空运输的决策变量

除卡车运输外，还可进行航空运输，用 $c_{ij,1}$ 表示公司一从城市 i 到城市 j 的航空运输量，其也是影响最终运输成本的因子之一。

（二）目标函数

本题目标为最小化运输成本，而运输成本由卡车运输与航空运输的成本构成，故而得到下式目标函数：

$$\min \sum_n \sum_i \sum_j a_{ij} x_{ij,n}^A + \sum_i \sum_j b_{ij} x_{ij,1}^B + C \sum_i \sum_j c_{ij,1} \quad (n = 1, 2)$$

（三）约束条件

1. 对卡车载货量的约束：

由卡车 A 的最大载货量为 12 吨，卡车 B 的最大载货量为 5 吨，且载货量不小于 0，得以下二式：

$$0 \leq x_{ij,n}^A q_{ij,n}^{A,1} \leq 12 \quad (n = 1, 2)$$

$$0 \leq x_{ij,1}^B q_{ij,1}^{B,1} \leq 5$$

2. 对各城市存、运货物数量的约束：

因假设公司一的货物运往小组一的城市，公司二的货物运往小组二的城市，而公司一所含货物量与小组一所需货物量相等，公司二所含货物量与小组二所含货物量相等，故公司一、二的货物需全部运出，分别送至小组一和小组二，得以下等式约束：

$$T_{mi} = \sum_j \sum_K q_{ij,n}^{K,1} x_{ij,n}^K - \sum_j \sum_K q_{ji,n}^{K,1} x_{ji,n}^K + \sum_j c_{ij,1}$$

$$D_{mj} = \sum_i \sum_K q_{ji,n}^{K,1} x_{ji,n}^K - \sum_i \sum_K q_{ij,n}^{K,1} x_{ij,n}^K + \sum_i c_{ij,1}$$

$$(m = 1, 2; K = A, B; \text{当 } K = A \text{ 时}, n = 1, 2; \text{当 } K = B \text{ 时}, n = 1)$$

3. 对卡车运输的约束：

$x_{ij,n}^K$ 表示第 n 辆卡车 K 是否参与 i 地到 j 地的货物运输，为 0-1 变量，即：

$$x_{ij,n}^K \in \{0, 1\}$$

具体表示为 $x_{ij,n}^K = \begin{cases} 1, & \text{第 } n \text{ 辆卡车 } K \text{ 参与了 } i \text{ 地到 } j \text{ 地的货物运输} \\ 0, & \text{第 } n \text{ 辆卡车 } K \text{ 没参与 } i \text{ 地到 } j \text{ 地的货物运输} \end{cases}$

4. 对航空运输的约束：

航空运输的数量必定为正值，故得到如下约束：

$$z_{ij} \geq 0$$

综合决策变量、目标函数、约束条件三点，可将所求问题转化为以下模型：

$$\min \sum_n \sum_i \sum_j a_{ij} x_{ij,n}^A + \sum_i \sum_j b_{ij} x_{ij,1}^B + C \sum_i \sum_j c_{ij,1} \quad (n = 1, 2)$$

$$s. t. \left\{ \begin{array}{l} 0 \leq x_{ij,1}^B q_{ij,1}^{B,1} \leq 5 \\ 0 \leq x_{ij,n}^A q_{ij,n}^{A,1} \leq 12 \quad (n = 1, 2) \\ T_{mi} = \sum_j \sum_K q_{ij,n}^{K,1} x_{ij,n}^K - \sum_j \sum_K q_{ji,n}^{K,1} x_{ji,n}^K + \sum_j c_{ij,1} \\ D_{mj} = \sum_i \sum_K q_{ij,n}^{K,1} x_{ij,n}^K - \sum_i \sum_K q_{ji,n}^{K,1} x_{ji,n}^K + \sum_i c_{ij,1} \\ x_{ij,n}^K \in \{0, 1\} \\ c_{ij,1} \geq 0 \end{array} \right.$$

($m = 1, 2$; $K = A, B$; 当 $K = A$ 时, $n = 1, 2$; 当 $K = B$ 时, $n = 1$)

公司二的处理方法相同，仅改变初始数据。

三、将优化模型转化为 QUBO 模型

对于目标函数中的变量 $x_{ij,n}^A$ 与 $x_{ij,n}^B$ ，其已为二进制变量，无需进行转化；而 c_{ij} 为非二进制变量，故对其进行离散化处理，使其变为二进制变量。

对于约束条件

$$\left\{ \begin{array}{l} 0 \leq x_{ij,1}^B q_{ij,1}^{B,1} \leq 5 \\ 0 \leq x_{ij,n}^A q_{ij,n}^{A,1} \leq 12 \quad (n = 1, 2) \\ T_{mi} = \sum_j \sum_K q_{ij,n}^{K,1} x_{ij,n}^K - \sum_j \sum_K q_{ji,n}^{K,1} x_{ji,n}^K + \sum_j c_{ij} \\ D_{mj} = \sum_i \sum_K q_{ij,n}^{K,1} x_{ij,n}^K - \sum_i \sum_K q_{ji,n}^{K,1} x_{ji,n}^K + \sum_i c_{ij} \\ x_{ij,n}^K \in \{0, 1\} \\ c_{ij} \geq 0 \end{array} \right.$$

对于等式约束，可将其移到等式一边，再转化为平方形式，将转化后的等价约束乘以一个较大的惩罚量 P 加入目标函数中。对于不等式约束，首先向不等式约束中引入松弛变量将其转化为等式约束，而后同上述方法处理。

为了使记法与应用程序保持一致，本文首先将对于变量进行重新编号，具体如下：

$$(x_{12,1}^A, x_{12,2}^A, x_{12,1}^B, \dots, x_{65,1}^B) = (x_1, x_2, \dots, x_{90})$$

故 QUBO 模型的转化如下：

$$\begin{aligned}
\min & \sum_n \sum_i \sum_j a_{ij} x_{ij,n}^A + \sum_i \sum_j b_{ij} x_{ij,1}^B + C \sum_i \sum_j c_{ij,1} \\
& - \sum_{i=1,2,6} P \left(\sum_j \sum_K q_{ij,n}^{K,1} x_{ij,n}^K - \sum_j \sum_K q_{ji,n}^{K,1} x_{ji,n}^K + \sum_j c_{ij} - T_{mi} \right)^2 \\
& - \sum_{j=3,4,5} P \left(\sum_j \sum_K q_{ij,n}^{K,1} x_{ij,n}^K - \sum_j \sum_K q_{ji,n}^{K,1} x_{ji,n}^K + \sum_j c_{ij} - D_{mj} \right)^2 \\
& - \sum_i \sum_j P (x_{ij,n}^A q_{ij,n}^A + x_{91} + 2x_{92} + 4x_{93} - 12)^2 \\
& - \sum_i \sum_j P (x_{ij,n}^B q_{ij,n}^B + x_{94} + 2x_{95} - 5)^2
\end{aligned}$$

又因 $x = \{0,1\}$ ，故有 $x^2 = x$ ，

故上式可转化为 QUBO 模型的标准形式：

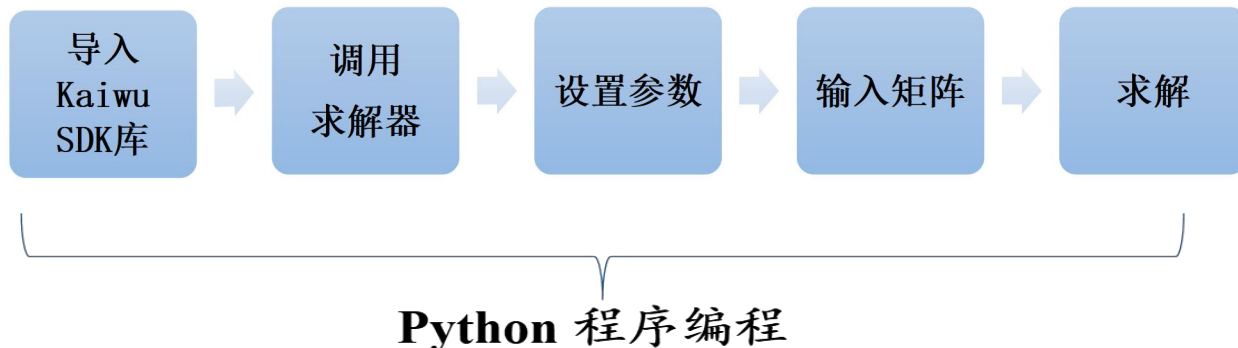
$$\min x^T Q x + c_0$$

其中 c_0 为一常数。

5.1.3 模型的求解

(1) 调用 CIM 模拟器和模拟退火求解器

使用 CIM 模拟器和模拟退火求解器对 QUBO 模型问题的求解思路大致相同，如图所示：



通过程序运行得到结果，对比得两种求解器所得结果相同，结果如表 4、表 5 所示：

表 4 公司一最优方案

公司一最优货车租赁方案和货物运输方案（吨）							
收货地 发货地	上海	西安	昆明	深圳	天津	郑州	成本（元）
上海				A(12), A(7)			73000
西安			A(12), A(7)	飞(1)			87000
昆明							
深圳							
天津							
郑州				B(5)	A(7)		37500
总成本（元）	197500						

表 5 公司二最优方案

公司二最优货车租赁方案和货物运输方案（吨）							
收货地 发货地	上海	西安	昆明	深圳	天津	郑州	成本（元）
上海			飞（1）	A(12)	飞（1）		56500
西安			A(12), A(12)				77000
昆明							
深圳			B(2)				17500
天津							
郑州					A(12), A(6)		21000
总成本（元）	172000						

以上两表格中，A(a)表示通过 A 货车运载 a 吨货物，从发货地发往收货地；B(b)表示通过 B 货车运载 b 吨货物，从发货地发往收货地；飞(c)表示通过航空运输的方式运载 c 吨货物，从发货地发往收货地。具体代码实现见附录 3。

（2）贪心算法

贪心算法的基本思路是从问题的某一个初始解出发一步一步地进行，根据某个优化测度，每一步都要确保能获得局部最优解。每一步只考虑一个数据，其选取应该满足局部优化的条件。若下一个数据和部分最优解连在一起不再是可行解时，就不把该数据添加到部分解中，直到把所有数据枚举完，或者不能再添加算法停止。^[4]

本问题运用贪心算法的基本步骤如下：

- Step1) 初始化： 创建两个列表，分别存储公司一和公司二的运输方案，初始为空。
- Step2) 循环遍历： 遍历所有可能的运输路径，计算每条路径的运输成本。
- Step3) 选择最优路径： 在每轮循环中，选择运输成本最低的路径作为当前公司的运输方案，并将该路径的成本累加到总成本中。
- Step4) 更新剩余货物量： 更新出发城市和到达城市的剩余货物量。
- Step5) 重复步骤 2-4： 直到所有货物都被运输完毕。
- Step6) 输出结果： 输出公司一和公司二的运输方案以及总成本。

该算法（具体实现见附录 6）所得最终结果与（1）中调用 CIM 模拟器和模拟退火求解器所得结果一致。

（3）遗传算法

遗传算法是模拟达尔文生物进化论的自然选择和遗传学机理的生物进化过程的计算模型，是一种通过模拟自然进化过程搜索最优解的方法。该算法通过数学的方式,利用计算机仿真运算,将问题的求解过程转换成类似生物进化中的染色体基因的交叉、变异等过程。在求解较为复杂的组合优化问题时,相较于常规的优化算法,通常能够较快地获得较好的优化结果^[5]。

本问题运用遗传算法的基本步骤如下：

- Step1) 编码： 将运输方案编码成染色体，例如每个染色体可以表示为 $[i, j, k, n]$ ，其中 i 和 j 分别表示出发城市和到达城市， k 和 n 的组合分别表示公司一和公司二使用的卡车类型。
- Step2) 初始种群： 随机生成一定数量的初始种群，每个种群代表一个运输方案。
- Step3) 适应度函数： 定义适应度函数，计算每个染色体的运输成本，运输成本越低，适应度越高。
- Step4) 选择： 根据适应度函数，选择一定数量的优秀染色体作为下一代种群的父代。

Step5) 交叉：对父代染色体进行交叉操作，产生新的染色体。

Step6) 变异：对新的染色体进行变异操作，增加种群的多样性。

Step7) 迭代：重复步骤 4-6，直到达到终止条件，例如迭代次数达到上限或适应度函数值不再提升。

该算法（具体实现见附录 7）所得最终结果与（1）中调用 CIM 模拟器和模拟退火求解器所得结果一致。

5.1.4 问题一模型检验

一、各种求解方法之间的比较

表 6 公司一最小总成本求解结果比较			
求解器	最小总成本	迭代次数	迭代时间
CIM 求解器	197500	10	25.765423s
模拟退火求解器	197500	10	24.369785s
贪心算法	197500	15	60.4605s
遗传算法	197500	20	80.4964s

表 7 公司二最小总成本求解结果比较			
求解器	最小总成本	迭代次数	迭代时间
CIM 求解器	172000	10	23.469723s
模拟退火求解器	172000	10	24.134389s
贪心算法	172000	15	59.7645s
遗传算法	172000	20	81.6794s

二、QUBO 模型与非量子化算法的对比

根据所列出的最优化问题的目标函数与约束条件，本文还使用遗传算法及贪心算法进行求解（具体实现见附录 6、7），将这两者优化方案与转化为 QUBO 模型利用量子计算技术所得结果进行比较，发现后者的运算速度高于前者，故证明了此题所运用的 QUBO 模型的优越性。具体表现中的优缺点如表格所示：

	QUBO 模型	遗传算法	贪心算法
优点	计算速度快、精度高	有较强的全局搜索能力	时间复杂度低
缺点	技术潜力仍待挖掘	大规模问题求解受限	可能陷入局部最优
优化效果比较	QUBO 模型>遗传算法>贪心算法		

5.2 问题二模型的建立与求解

5.2.1 问题二模型建立

一、模型抽象及简化

1. 抽象为数学模型

具体符号及字母的设置同问题一中做法。

2. 对 $x_{ij,n}^K$ 的分析

(1) $x_{ij,n}^K$ 表示货车 K 经最优路线由*i*城市发出到达*j*城市，使得运输成本达到最低，由最短路径矩阵可知，该路线可能会途径其他城市*v*，但该模型认为 K 货车在此运输过程不在*v*城市做停留。

(2) 对 *n* 的分析

① $K = A$ 时

由于该小问中允许公司间拼货形式的存在，故两公司为合作关系，可共同完成小组一及小组二中所需货物量的运输，计算的出货单个到达城市中对于货量的最大需求为 46 吨，卡车 A 的最大载重量为 12 吨， $4 \times 12 = 48 > 46$ ，故使用卡车 A 的最大数量为 4，即此时 $n = 1, 2, 3, 4$ 。

② $K = B$ 时

与问题一中分析相同，即此时 $n = 1$ 。

二、构建优化模型

(一) 决策变量

与问题一的决策变量相同，即卡车运输的二进制决策变量 $x_{ij,n}^K$ 与航空运输的决策变量及航空运输量 $c_{ij,M}$ ，其中 $i, j \in \{1, 2, 3, 4, 5, 6\}$, $m = 1, 2$; $K = A, B$; 当 $K = A$ 时, $n = 1, 2, 3, 4$; 当 $K = B$ 时, $n = 1$ 。

(二) 目标函数:

最终目标是使得两公司的运输总成本最低，即目标函数为:

$$\min \sum_n \sum_i \sum_j a_{ij} x_{ij,n}^{A,M} + \sum_i \sum_j b_{ij} x_{ij,1}^B + C \sum_M \sum_i \sum_j c_{ij,M}$$

(三) 约束条件:

1. 对卡车运载货物数量的约束:

$$\begin{aligned} 0 \leq \sum_M x_{ij,n}^A q_{ij,n}^{A,M} &\leq 12 \quad (n = 1, 2, 3, 4; M = 1, 2) \\ 0 \leq \sum_M x_{ij,1}^B q_{ij,1}^{B,M} &\leq 5 \quad (M = 1, 2) \end{aligned}$$

(其中 $q_{ij,n,m}^{K,M}$ 表示公司 M 从城市*i*到城市*j*通过卡车 K 所运载的的货物数量)

2. 各城市存、运货物数量约束:

$$\begin{aligned} T_{mi} &= \sum_j \sum_K q_{ij,n}^{K,M} x_{ij,n}^K - \sum_j \sum_K q_{ji,n}^{K,M} x_{ji,n}^K + \sum_j c_{ij,M} \\ D_{mj} &= \sum_i \sum_K q_{ij,n}^{K,M} x_{ij,n}^K - \sum_i \sum_K q_{ji,n}^{K,M} x_{ji,n}^K + \sum_i c_{ij,M} \\ (n &= 1, 2, 3, 4; M = 1, 2) \end{aligned}$$

T_{Mi} 表示公司M在出发城市*i*的初始货物数量(此处 $i = 1, 2, 6$); D_{Mj} 表示公司M需运往到达城市*j*的货物数量(此处 $j = 3, 4, 5$)。

其余对卡车运输的约束及对航空运输的约束同问题一。

综合决策变量、目标函数、约束条件三点，可将所求问题转化为以下模型：

$$\begin{aligned}
 \min & \sum_n \sum_i \sum_j a_{ij} x_{ij,n}^{A,M} + \sum_i \sum_j b_{ij} x_{ij,1}^B + C \sum_M \sum_i \sum_j c_{ij,M} \\
 \text{s.t.} & \begin{cases} 0 \leq \sum_M x_{ij,n}^A q_{ij,n}^{A,M} \leq 12 & (n = 1, 2, 3, 4; M = 1, 2) \\ 0 \leq \sum_M x_{ij,1}^B q_{ij,1}^{B,M} \leq 5 & (M = 1, 2) \\ T_{mi} = \sum_j \sum_K q_{ij,n}^{K,M} x_{ij,n}^K - \sum_j \sum_K q_{ji,n}^{K,M} x_{ji,n}^K + \sum_j c_{ij,M} \\ D_{mj} = \sum_i \sum_K q_{ij,n}^{K,M} x_{ij,n}^K - \sum_i \sum_K q_{ji,n}^{K,M} x_{ji,n}^K + \sum_i c_{ij,M} \\ x_{ij,n}^K \in \{0, 1\} \\ c_{ij,M} \geq 0 \end{cases} \\
 & (m = 1, 2; K = A, B; \text{当 } K = A \text{ 时}, n = 1, 2, 3, 4; \text{当 } K = B \text{ 时}, n = 1)
 \end{aligned}$$

三、转化为 QUBO 模型

处理过程与上一问相同，首先，为使记法与应用程序保持一致，首先对于变量进行重新编号，具体如下：

$$(x_{12,1}^A, x_{12,2}^A, x_{12,3}^A, x_{12,4}^A, x_{12,1}^B, \dots, x_{65,1}^B) = (x_1, x_2, \dots, x_{150})$$

接着向不等式约束中加松弛变量转化为等式约束，再将等式约束乘以惩罚项加入目标函数中，转化得：

$$\begin{aligned}
 \min & \sum_n \sum_i \sum_j a_{ij} x_{ij,n}^{A,M} + \sum_i \sum_j b_{ij} x_{ij,1}^B + C \sum_m \sum_i \sum_j c_{ij,m} \\
 & - \sum_{i=1,2,6} P \left(\sum_j \sum_K q_{ij,n}^{K,M} x_{ij,n}^K - \sum_j \sum_K q_{ji,n}^{K,M} x_{ji,n}^K + \sum_j c_{ij,M} - T_{mi} \right)^2 \\
 & - \sum_{j=3,4,5} P \left(\sum_j \sum_K q_{ij,n}^{K,M} x_{ij,n}^K - \sum_j \sum_K q_{ji,n}^{K,M} x_{ji,n}^K + \sum_j c_{ij,M} - D_{mj} \right)^2 \\
 & - \sum_i \sum_j P (x_{ij,n}^A q_{ij,n}^{A,M} + x_{151} + 2x_{152} + 4x_{153} - 12)^2 \\
 & - \sum_i \sum_j P (x_{ij,n}^B q_{ij,n}^{B,M} + x_{154} + 2x_{155} - 5)^2
 \end{aligned}$$

又由 $x^2 = x$ ，可将模型转化为 QUBO 标准形式，即：

$$\min x^T Q x + c_0$$

此时由于最后建立的 QUBO 模型比特数较高，故考虑将一个较大的 QUBO 模型分解成多个子模型 (subQUBO) 进行求解，这是一种用于提高优化算法效率和可解性的策略^[6]。

subQUBO 模型具体求解步骤如下：

Step1)将原问题的 QUBO 模型分解成多个 subQUBO 模型每个 subQUBO 模型只包含一部分的决策变量和约束条件。

Step2)定义 subQUBO 模型的决策变量、约束条件、目标函数。

Step3)使用 Kaiwu SDK 的模拟退火求解器和 CIM 模拟器来求解每个 subQUBO 模型。

Step4)更新原问题的解，根据每个 subQUBO 模型的最优解，更新原问题的解。

Step5)重复步骤 2 至步骤 4，直到所有 subQUBO 模型都被求解并更新了原问题的解。

5.2.2 问题二模型求解

(1)调用 CIM 模拟器和模拟退火求解器

利用 Kaiwu SDK 库中的 CIM 模拟器和模拟退火求解器进行求解，得到结果如表格 8 所示：

表 8 两公司合作运营时最优方案

两公司合作运营时最优的货车租赁方案和货物运输方案（吨）							
收货地 发货地	上海	西安	昆明	深圳	天津	郑州	成本（元）
上海				A(12, 0), A(7, 5)		A(0, 9)	91500
西安			A(12, 0), A(7, 3) A(0, 12), A(0, 12)			B(1, 0)	158000
昆明							
深圳							
天津							
郑州		B(0, 3)		A(6, 5)	A(0, 12), A(7, 5) B(0, 2)		68500
总成本（元）	318000						

上表中，A(a1, a2)表示通过 A 货车运载一公司 a1 吨货物，二公司 a2 吨货物，从发货地发往收货地；B(b1, b2)表示通过 B 货车运载一公司 b1 吨货物，二公司 b2 吨货物，从发货地发往收货地；飞(c1, c2)表示通过航空运输的方式运载一公司 c1 吨货物，二公司 c2 吨货物，从发货地发往收货地。

与问题一中所得方案相比，此问题中两公司合作运营时的方案节约总成本为：

$$197500 + 172000 - 318000 = 51500 \text{（元）}$$

(2)贪心算法

问题二可以分解为以下三个子问题：

- ①路径选择问题：选择最优的运输路径，使得运输成本最低。
- ②货物分配问题：将货物合理分配到每辆卡车上，确保卡车装载量不超过其最大载货量。
- ③拼货方案问题：确定两公司之间如何进行拼货，使得总成本最低。

此时贪心算法求解思路：

- ①针对路径选择问题：使用贪心算法选择最优的运输路径，参考问题一的贪心算法代码。
- ②针对货物分配问题：对于每条路径，使用贪心算法将货物分配到卡车上，优先分配载货量大的卡车。
- ③针对拼货方案问题：对于每条路径，比较两公司单独运输和拼货运输的成本，选择成本更低的方案。

(3) 遗传算法

同（2）将该问题分解为三个子问题。本问题的遗传算法求解思路：

Step1) 编码： 将运输方案编码成染色体，例如每个染色体可以表示为 $[i, j, k, n, p]$ ，其中 i 和 j 分别表示出发城市和到达城市， k 和 n 的组合分别表示公司一和公司二使用的卡车类型， p 表示是否拼货（0 表示不拼货，1 表示拼货）。

Step2) 初始种群： 随机生成一定数量的初始种群，每个种群代表一个运输方案。

Step3) 适应度函数： 定义适应度函数，计算每个染色体的运输成本，运输成本越低，适应度越高。

Step4) 选择： 根据适应度函数，选择一定数量的优秀染色体作为下一代种群的父代。

Step5) 交叉： 对父代染色体进行交叉操作，产生新的染色体。

Step6) 变异： 对新的染色体进行变异操作，增加种群的多样性。

Step7) 迭代： 重复步骤 4-6，直到达到终止条件，例如迭代次数达到上限或适应度函数值不再提升。

5.2.3 问题二模型检验

该小问同样使用贪心算法及遗传算法进行求解（具体实现见附录 10、11），将这两种算法所得结果及迭代效果与用 CIM 求解器及模拟退火求解器得到的结果进行比较，结果如表 9 所示：

表 9 两公司最小总成本求解结果比较

求解器	最小总成本	迭代次数	迭代时间
CIM 求解器	318000	30	55.526729s
模拟退火求解器	318000	30	53.945355s
贪心算法	318000	53	150.4113s
遗传算法	318000	55	183.5762s

可得使用 CIM 求解器及模拟退火求解器的求解最终结果与贪心算法及遗传算法的结果相同，但在迭代次数及迭代时间上展现出其优越性，故认为该模型良好。

5.3 问题三模型

5.3.1 问题背景及实际意义

在银行信用卡或相关的贷款等业务中，对客户授信之前，需要先通过各类审核规则对客户的信用等级进行评定，只有通过评定的客户才能获得信用卡或贷款资格。而该审核规则过程实际是经过一重或者多重组合规则后对客户进行评分，将其称为信用评分卡。每个信用评分卡有多种阈值设置（但其中有且仅有一个阈值生效），不同的信用评分卡在不同的阈值下，对应着不同的通过率和坏账率。一般通过率越高，坏账率也会越高，反之，通过率越低，坏账率越低。^[7]

对银行来说，通过率越高，通过贷款资格审核的客户数量就越多，相应的银行理论上从客户手中获得的利息收入就会越多，但高通过率就意味着高坏账率，而坏账意味着放贷资金损失的风险，因此银行最终的收入可以定义为：

$$\text{最终收入} = \text{贷款利息收入} - \text{坏账损失}$$

下表为查询资料^[8]所得的部分信用评分卡及其对应的 10 个阈值，以及相应阈值对应的不同的坏账率和通过率的表格：

信用评分卡 1			信用评分卡 2			信用评分卡 3		
阈值	通过率	坏账率	阈值	通过率	坏账率	阈值	通过率	坏账率
1	6%	0.60%	1	7%	0.55%	1	4%	0.65%
2	13%	1.20%	2	12%	1.10%	2	14%	1.20%
3	26%	1.70%	3	24%	1.60%	3	22%	1.65%
4	37%	2.30%	4	35%	2.10%	4	34%	2.15%
5	44%	2.60%	5	47%	2.30%	5	45%	2.65%
6	52%	3.10%	6	54%	2.60%	6	53%	3.25%
7	63%	3.60%	7	68%	3.30%	7	66%	3.65%
8	76%	4.10%	8	71%	4.20%	8	74%	4.20%
9	81%	4.60%	9	86%	4.80%	9	85%	4.60%
10	93%	5.20%	10	91%	5.10%	10	97%	5.05%

本问的目标在 m 张信用评分卡中找出 1 张及一定的阈值,使得银行的借款业务得到最高收入。

5.3.2 符号设计

符号	说明	单位
A	贷款资金	元
q	银行贷款利息收入率	/
A_i	信用卡 i 的通过率	/
B_i	信用卡 i 的坏账率	/
x_i	是否采用第 i 张信用卡（是取 1，不是取 0）	/

5.3.3 模型建立与求解

对于该问题,首先将问题等价转化为从 $10m$ 张信用评分卡中找出一张信用卡,即将原问题中一张信用卡对应十个阈值转化为每一张信用评分卡对应一个阈值,从而实现模型的简化。将银行的最终收入转化为数学表达式得:

$$\begin{aligned}
& Aq \sum_{i=1}^{10m} A_i x_i (1 - \sum_{i=1}^{10m} B_i x_i) - A \sum_{i=1}^{10m} A_i x_i \sum_{i=1}^{10m} B_i x_i \\
&= \sum_{i=1}^{10m} (A_i x_i) \left(Aq - (A + Aq) \sum_{i=1}^{10m} B_i x_i \right) \\
&= -(A + Aq) \sum_{i=1}^{10m} B_i x_i \sum_{i=1}^{10m} A_i x_i + Aq \sum_{i=1}^{10m} A_i x_i
\end{aligned}$$

本问目标是求得上式的最大值以达到最大利益,将上式取负后转化为求最小值问题,得到目标函数为:

$$(A + Aq) \sum_{i=1}^{10m} B_i x_i \sum_{i=1}^{10m} A_i x_i - Aq \sum_{i=1}^{10m} A_i x_i$$

由于最终只取唯一一张信用卡，故得到约束条件为：

$$\sum_{i=1}^{10m} x_i = 1$$

该约束条件等价于：

$$\left(\sum_{i=1}^{10m} x_i - 1 \right)^2 = 0$$

转化为 QUBO 表达式为：

$$\min (A + Aq) \sum_{i=1}^{10m} B_i x_i \sum_{i=1}^{10m} A_i x_i - Aq \sum_{i=1}^{10m} A_i x_i + P \left(\sum_{i=1}^{10m} x_i - 1 \right)^2$$

其中 P 为惩罚项。

为了计算该模型中的比特数量级，我们需要确定模型中涉及的变量数量。根据模型假设，我们有 m 张信用评分卡，每张卡有 10 个阈值。因此，我们有 10m 个可能的阈值选择。因此：**比特数量级 = 210m**。

六、模型评价、改进与推广

6.1 模型的优点

1. 本文在构建优化模型前运用 Dijkstra 算法更新最短路径，实现更优，并用 Floyd 算法确定了其正确性。
2. 通过预处理数据分析得出卡车的最大数量，实现模型的简化。
3. 问题求解效率高: QUBO 模型能够利用量子计算的优势，在短时间内求解复杂优化问题，相较于传统算法具有显著的速度优势。
4. 结果精度高: QUBO 模型能够得到全局最优解或近似最优解，避免陷入局部最优，从而保证结果的精度和可靠性。
5. 模型适用性强: QUBO 模型可以应用于各种类型的组合优化问题，具有广泛的适用性。
6. 易于实现: 利用 Kaiwu SDK 等开发工具，可以方便地构建和求解 QUBO 模型，降低了量子计算技术的应用门槛。
7. 本文还通过贪心算法和遗传算法对 CIM 模拟器和模拟退火求解器的求解结果进行了比较验证，进一步证明了 QUBO 模型的优越性。

6.2 模型的缺点

1. 实际航空运输中的成本受距离的影响较大，而本文为简化模型将其设为一固定值。
2. 实际航空运输成本简化：模型中将航空运输成本设为固定值，未考虑距离因素，与实际情况存在偏差。
3. 模型复杂度限制：当问题规模较大时，QUBO 模型中的 Q 矩阵复杂度过高时会使算力减小，QUBO 模型的构建和求解可能会面临一定的挑战，需进一步采取 subQUBO 模型的分解。
4. 量子计算硬件限制：目前量子计算机的规模和稳定性有限，限制了 QUBO 模型的实际应用范围。

6.3 模型的改进与推广

- 1、改进之处主要包括以下几点：
 - (1) 航空运输成本线性化：可以根据距离对航空运输成本进行线性化处理，使模型更接近实际情况。
 - (2) 考虑时间窗约束：可以在模型中加入时间窗约束，考虑卡车运输的时间限制，使模型更加完善。
 - (3) 考虑车辆维修保养：可以在模型中加入车辆维修保养成本，使模型更加全面。
- 2、模型推广主要可以从以下几点考虑：
 - (1) 物流领域：将模型应用于其他物流优化问题，如路径规划、库存管理、运输调度等。
 - (2) 金融领域：将模型应用于金融优化问题，如投资组合优化、信用评级、风险管理等。
 - (3) 其他领域：将模型应用于其他需要解决组合优化问题的领域，如生产调度、人员排班等。

七、参考文献

- [1] Glover, F., Kochenberger, G., Hennig, R. *et al.* Quantum bridge analytics I: a tutorial on formulating and using QUBO models. *Ann Oper Res* **314**, 141–183 (2022).
- [2] Kirkpatrick, S., Gelatt Jr, C. D. Vecchi, M. P. Optimization by simulated annealing. *Science*, 220(4598), 671-680(1983).
- [3] Sunshing. 优化算法——模拟退火算法 (Simulated Annealing, SA)
https://blog.csdn.net/weixin_52721512/article/details/126497778/ 2024 年 4 月 24 日
- [4] 常友渠,肖贵元,曾敏.贪心算法的探讨与研究[J].重庆电力高等专科学校学报,2008,13(3):40-424
- [5] 郑树泉. 工业智能技术与应用[M]. 上海. 上海科学技术出版社. 2019. 250-251
- [6] Dury B, Di Matteo O. A QUBO formulation for qubit allocation[J]. arXiv preprint arXiv: 2009.00140, 2020.
- [7] 陈粘. 高维金融市场的时变投资组合优化研究[D].成都理工大学,2021.
- [8] 汪勇,孟香君,沈维萍.量子计算在经济与金融领域中的应用[J].经济学动态,2023(01):126-143.

八、附录

附录 1 Dijkstra 算法得到最短路径的代码（以 A 车为例）.py

```
import matplotlib.pyplot as plt
import networkx as nx
from matplotlib.font_manager import FontProperties

# 定义城市名称和位置
cities = {
    "上海": (0, 1),
    "西安": (0.866, 0.5),
    "昆明": (0.866, -0.5),
    "深圳": (0, -1),
    "天津": (-0.866, -0.5),
    "郑州": (-0.866, 0.5)
}

# 手动指定节点的位置，确保六边形布局且不交叉
pos = {
    "上海": (1, -0.5),
    "西安": (-0.5, 0.5),
    "昆明": (-0.5, -0.5),
    "深圳": (0.25, -1),
    "天津": (1, 0.5),
    "郑州": (0.25, 1)
}

# 权重矩阵
weights = [
    [0, 13500, 26500, 16500, 13500, 8500],
    [13500, 0, 18500, 20500, 10500, 3500],
    [26500, 18500, 0, 7500, 31500, 24500],
    [16500, 20500, 7500, 0, 25500, 17500],
    [13500, 10500, 31500, 25500, 0, 5500],
    [8500, 3500, 24500, 17500, 5500, 0]
]

# 创建无向图
G = nx.Graph()

# 添加节点和带权重的边
for city, position in cities.items():
    G.add_node(city, pos=(pos[city][0], pos[city][1]))

for i in range(len(weights)):
    for j in range(i + 1, len(weights[i])):
        if i < j:
            G.add_edge(list(cities.keys())[i], list(cities.keys())[j], weight=weights[i][j])

# 计算任意两点间的最短路径
```



```

all_pairs_shortest_paths = dict(nx.all_pairs_dijkstra_path(G))

# 输出上海、西安、郑州到昆明、深圳、天津的最短路径
start_cities = ["上海", "西安", "郑州"]
end_cities = ["昆明", "深圳", "天津"]

for start in start_cities:
    for end in end_cities:
        if start != end:
            shortest_path = all_pairs_shortest_paths[start][end]
            path_length = nx.dijkstra_path_length(G, source=start, target=end,
weight='weight')
            print(f'从 {start} 到 {end} 的最短路径为: {'->'.join(shortest_path)}, 权重为
{path_length} .')

# 设置中文字体
plt.rcParams['font.family'] = 'SimHei'

# 绘制图形
plt.figure(figsize=(10, 8))

# 绘制节点
nx.draw_networkx_nodes(G, pos, node_size=700, node_color='skyblue')

# 绘制最短路径上的边和标签
for source in all_pairs_shortest_paths:
    for target in all_pairs_shortest_paths[source]:
        path = all_pairs_shortest_paths[source][target]
        if len(path) > 1:
            path_edges = list(zip(path[:-1], path[1:]))
            nx.draw_networkx_edges(G, pos, edgelist=path_edges, width=2.0, alpha=0.7)
            edge_labels = {(path[i], path[i+1]):
weights[list(cities.keys()).index(path[i])][list(cities.keys()).index(path[i+1])] for i in
range(len(path) - 1)}
            nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels,
font_color='red', font_family='SimHei')

# 绘制节点的标签
nx.draw_networkx_labels(G, pos, font_size=12, font_family='SimHei')

plt.title('卡车 A 单趟成本 最短路径', fontsize=15, fontfamily='SimHei')
plt.axis('off')
plt.show()

```

附录 2 Floyd 算法得到最短路径的代码（以 A 车为例）.m

```

function floydWarshallExample()
    function [D, R] = floydWarshall(distanceMatrix)
        % distanceMatrix: 无向图的距离矩阵，对角线元素为 0，非直接相连的顶点间距离
为 Inf
        n = size(distanceMatrix, 1); % 顶点数量
        D = distanceMatrix; % 初始化 D 矩阵
    end
end

```

```

R = zeros(n); % 初始化 R 矩阵, 用于记录最短路径的前驱顶点

% Floyd 算法
for k = 1:n
    for i = 1:n
        for j = 1:n
            if D(i, k) + D(k, j) < D(i, j)
                D(i, j) = D(i, k) + D(k, j); % 更新最短距离
                R(i, j) = k; % 更新前驱顶点
            end
        end
    end
end

A = [0 28500 56500 36500 28500 18500;
     28500 0 38500 40500 20500 8500;
     56500 38500 0 17500 61500 49500;
     36500 40500 17500 0 50500 37500;
     28500 20500 61500 50500 0 10500;
     18500 8500 49500 37500 10500 0];

B = [0 20000 42000 26000 20000 12000;
     20000 0 28000 30000 14000 4000;
     42000 28000 0 11000 47000 37000;
     26000 30000 11000 0 38000 27000;
     20000 14000 47000 38000 0 6000;
     12000 4000 37000 27000 6000 0];

[DA, RA] = floydWarshall(A);
[DB, RB] = floydWarshall(B);

% 输出结果
disp('卡车 A 的最短路径距离矩阵 DA:');
disp(DA);
disp('卡车 A 的最短路径前驱顶点矩阵 RA:');
disp(RA);

disp('卡车 B 的最短路径距离矩阵 DB:');
disp(DB);
disp('卡车 B 的最短路径前驱顶点矩阵 RB:');
disp(RB);
end

```

附录 3 构造问题一中 QUBO 模型的 Q 矩阵. py

```
import numpy as np
```

```
# 定义城市
```

```

J = ["上海", "西安", "昆明", "深圳", "天津", "郑州"]
n = len(J) # 城市数量
# 定义卡车类型
K = ["A", "B"]
m = len(K) # 卡车类型数量
# 定义决策变量索引
idx = {}
idx_air = {}
current_idx = 0
# 生成卡车运输决策变量索引
for k in K:
    for i in [1, 2, 6]:
        for j in [3, 4, 5]:
            if i != j:
                for l in range(2 if k == "A" else 1):
                    idx_key = f"{k}{i}{j}{l}"
                    idx[idx_key] = current_idx
                    current_idx += 1
# 生成航空运输决策变量索引
for i in [1, 2, 6]:
    for j in [3, 4, 5]:
        if i != j:
            idx_air_key = f"{i}{j}"
            idx_air[idx_air_key] = current_idx
            current_idx += 1
# 定义卡车 A 和 B 的最短路径矩阵
distances_A = np.array([
    [0, 13500, 26500, 16500, 13500, 8500],
    [13500, 0, 18500, 20500, 10500, 3500],
    [26500, 18500, 0, 7500, 31500, 24500],
    [16500, 20500, 7500, 0, 25500, 17500],
    [13500, 10500, 31500, 25500, 0, 5500],
    [8500, 3500, 24500, 17500, 5500, 0]
])
distances_B = np.array([
    [0, 11000, 24000, 14000, 11000, 6000],
    [11000, 0, 16000, 18000, 8000, 1000],
    [24000, 16000, 0, 5000, 29000, 22000],
    [14000, 18000, 5000, 0, 23000, 15000],
    [11000, 8000, 29000, 23000, 0, 3000],
    [6000, 1000, 22000, 15000, 3000, 0]
])
# 定义卡车参数
max_capacity = {"A": 12, "B": 5}
rent = {"A": 5000, "B": 3000}
# 定义航空运输成本
air_cost = 10000
# 定义初始货物量和目标货物量
initial_goods = {
    "1 上海": 9,

```

```

    "1 西安": 0,
    "1 郑州": 2,
    "2 上海": 4,
    "2 西安": 4,
    "2 郑州": 8
}
target_goods = {
    "1 昆明": 19,
    "1 深圳": 25,
    "1 天津": 7,
    "2 昆明": 27,
    "2 深圳": 10,
    "2 天津": 19
}
# 定义目标函数
linear = {}
quadratic = {}
# 卡车运输成本
for k in K:
    distances = distances_A if k == "A" else distances_B
    for i in [1, 2, 6]:
        for j in [3, 4, 5]:
            if i != j:
                for l in range(2 if k == "A" else 1):
                    idx_key = f"{k}{i}{j}{l}"
                    quadratic[idx[idx_key], idx[idx_key]] -= rent[k] * distances[i, j]
# 航空运输成本
for i in [1, 2, 6]:
    for j in [3, 4, 5]:
        if i != j:
            idx_air_key = f"{i}{j}"
            quadratic[idx_air[idx_air_key], idx_air[idx_air_key]] -= air_cost
# 卡车载货量约束
for k in K:
    for i in [1, 2, 6]:
        for j in [3, 4, 5]:
            if i != j:
                for l in range(2 if k == "A" else 1):
                    idx_key = f"{k}{i}{j}{l}"
                    linear[idx[idx_key]] -= max_capacity[k]
# 城市货物存运量约束
for i in [1, 2, 6]:
    for j in [3, 4, 5]:
        # 出发城市货物量
        for k in K:
            for l in range(2 if k == "A" else 1):
                idx_key = f"{k}{i}{j}{l}"
                quadratic[idx[idx_key], idx[idx_key]] += initial_goods[f"{i}{j}"]
        # 到达城市货物量
        for k in K:

```

```

        for l in range(2 if k == "A" else 1):
            idx_key = f"{k}{i}{j}{l}"
            quadratic[idx[idx_key], idx[idx_key]] -= target_goods[f"{i}{j}"]
# 卡车运输约束
for k in K:
    for i in [1, 2, 6]:
        for j in [3, 4, 5]:
            if i != j:
                for l in range(2 if k == "A" else 1):
                    for m in range(2 if k == "A" else 1):
                        idx_key1 = f"{k}{i}{j}{l}"
                        idx_key2 = f"{k}{i}{j}{m}"
                        if idx_key1 != idx_key2:
                            quadratic[idx[idx_key1], idx[idx_key2]] += 1
# 航空运输约束
for i in [1, 2, 6]:
    for j in [3, 4, 5]:
        if i != j:
            idx_air_key = f"{i}{j}"
            quadratic[idx_air[idx_air_key], idx_air[idx_air_key]] += 1

# 构建 QUBO 矩阵
Q = np.zeros((len(idx) + len(idx_air), len(idx) + len(idx_air)))

# 填充 QUBO 矩阵
for i in idx.keys():
    for j in idx.keys():
        Q[idx[i], idx[j]] = quadratic[i, j]
for i in idx_air.keys():
    for j in idx_air.keys():
        Q[idx_air[i], idx_air[j]] = quadratic[i, j]

```

附录 4 问题一调用 CIM 模拟器代码.py

```

import kaiwu as kw
import numpy as np

user_id = "用户 ID"
sdk_code = "SDK 授权码"

# 初始化 Kaiwu SDK
kw.license.init(user_id=user_id, sdk_code=sdk_code)

# 从附录 3 中获取 QUBO 矩阵
Q = np.zeros((len(idx) + len(idx_air), len(idx) + len(idx_air)))
# ... (此处省略构建 Q 矩阵的代码) ...

# 调用 Kaiwu SDK 的 CIM 模拟器
output = kw.cim.simulator(
    matrix=Q,
    pump=0.7,

```

```

        noise=0.01,
        laps=50,
        dt=0.1,
        normalization=0.3,
        iterations=10
    )

# 调用 Kaiwu SDK 的最优解采样器
opt = kw.sampler.optimal_sampler(matrix=Q, output=output, num_samples=0)

# 解析结果
best_solution = opt[0][0]
min_cost = (np.sum(Q) - np.dot(Q, best_solution).dot(best_solution)) / 4

# 输出结果
print(f"最小成本为: {min_cost} 元")
print(f"最优方案: {best_solution}")

```

附录 5 问题一调用模拟退火求解器代码.py

```

import kaiwu as kw
import numpy as np

user_id = "用户 ID"
sdk_code = "SDK 授权码"

# 初始化 Kaiwu SDK
kw.license.init(user_id=user_id, sdk_code=sdk_code)

# 从附录 3 中获取 QUBO 矩阵
Q = np.zeros((len(idx) + len(idx_air), len(idx) + len(idx_air)))
# ... (此处省略构建 Q 矩阵的代码) ...

# 设置模拟退火求解器参数
initial_temperature = 100
alpha = 0.99
cutoff_temperature = 0.001
iterations_per_t = 10
size_limit = 10

# 创建模拟退火求解器
sa_worker = kw.classical.SimulatedAnnealingOptimizer(
    initial_temperature=initial_temperature,
    alpha=alpha,
    cutoff_temperature=cutoff_temperature,
    iterations_per_t=iterations_per_t,
    size_limit=size_limit
)

# 调用模拟退火求解器
solution = sa_worker.solve(matrix=Q)

# 解析结果

```

```

best_solution = solution[0]
min_cost = (np.sum(Q) - np.dot(Q, best_solution).dot(best_solution)) / 4

# 输出结果
print(f'最小成本为: {min_cost} 元")
print(f'最优方案: {best_solution}")

```

附录 6 问题一贪心算法主要代码.py

```

#假定以下变量已从题干或之前代码求出：
# cost_matrix: 存储城市间运输成本的矩阵
# truck_capacity: 存储卡车载货量的列表
# truck_rent: 存储卡车日租金的列表
# initial_goods: 存储每个城市初始货物量的列表
# final_goods: 存储每个城市最终货物量的列表

def greedy_algorithm(cost_matrix, truck_capacity, truck_rent, initial_goods, final_goods):
    # 初始化运输方案和总成本
    solution_company1 = []
    solution_company2 = []
    total_cost = 0

    # 循环遍历所有城市
    for i in range(len(cost_matrix)):
        # 循环遍历所有卡车类型
        for truck_type in range(len(truck_capacity)):
            # 循环遍历所有可能的运输路径
            for j in range(len(cost_matrix[i])):
                # 如果当前路径可行，即出发城市有货物，到达城市需要货物
                if initial_goods[i] > 0 and final_goods[j] > 0:
                    # 计算当前路径的运输成本
                    transport_cost = cost_matrix[i][j]
                    rent_cost = truck_rent[truck_type] * (transport_cost /
truck_capacity[truck_type])
                    total_cost = transport_cost + rent_cost

                    # 如果当前路径的成本低于已知的最低成本，则更新方案
                    if total_cost < min_cost:
                        min_cost = total_cost
                        best_path = (i, j, truck_type)

                    # 更新出发城市和到达城市的剩余货物量
                    initial_goods[i] -= truck_capacity[truck_type]
                    final_goods[j] += truck_capacity[truck_type]

    # 将最优路径添加到运输方案中
    solution_company1.append(best_path)
    solution_company2.append(best_path)

```

```

    # 返回运输方案和总成本
    return solution_company1, solution_company2, total_cost

# 调用贪心算法求解最优解
solution_company1, solution_company2, total_cost = greedy_algorithm(cost_matrix,
truck_capacity, truck_rent, initial_goods, final_goods)

# 打印最优解
print("公司一运输方案:", solution_company1)
print("公司二运输方案:", solution_company2)
print("总成本:", total_cost)

```

附录 7 问题一遗传算法主要代码.py

```

import random

#假定以下变量已从题干或之前代码求出：
# cost_matrix: 存储城市间运输成本的矩阵
# truck_capacity: 存储卡车载货量的列表
# truck_rent: 存储卡车日租金的列表
# initial_goods: 存储每个城市初始货物量的列表
# final_goods: 存储每个城市最终货物量的列表

# 编码函数
def encode_solution(i, j, k, n):
    return [i, j, k, n]

# 解码函数
def decode_solution(solution):
    i, j, k, n = solution
    return i, j, k, n

# 适应度函数
def fitness_function(solution):
    i, j, k, n = decode_solution(solution)
    cost = cost_matrix[i][j] + truck_rent[k] * (cost_matrix[i][j] / truck_capacity[k])
    cost += cost_matrix[i][j] + truck_rent[n] * (cost_matrix[i][j] / truck_capacity[n])
    return 1 / cost

# 选择函数
def select(population, fitness):
    # 使用轮盘赌选择方法
    total_fitness = sum(fitness)
    cumulative_fitness = [sum(fitness[:i+1]) for i in range(len(fitness))]
    selected_indices = []
    for _ in range(len(population)):
        r = random.uniform(0, total_fitness)
        for i in range(len(cumulative_fitness)):
            if r < cumulative_fitness[i]:
                selected_indices.append(i)
                break

```



```

        return [population[i] for i in selected_indices]

# 交叉函数
def crossover(parent1, parent2):
    # 使用单点交叉方法
    r = random.randint(1, len(parent1) - 1)
    child1 = parent1[:r] + parent2[r:]
    child2 = parent2[:r] + parent1[r:]
    return child1, child2

# 变异函数
def mutate(solution, mutation_rate):
    if random.random() < mutation_rate:
        r1 = random.randint(0, len(solution) - 1)
        r2 = random.randint(0, len(solution) - 1)
        solution[r1], solution[r2] = solution[r2], solution[r1]
    return solution

# 遗传算法主函数
def genetic_algorithm(cost_matrix, truck_capacity, truck_rent, initial_goods, final_goods,
    population_size, mutation_rate, max_generations):
    # 初始化种群
    population = [encode_solution(random.randint(0, len(cost_matrix)-1), random.randint(0,
    len(cost_matrix[0])-1), random.randint(0, len(truck_capacity)-1), random.randint(0,
    len(truck_capacity)-1)) for _ in range(population_size)]

    for generation in range(max_generations):
        # 计算适应度
        fitness = [fitness_function(solution) for solution in population]

        # 选择
        selected_population = select(population, fitness)

        # 交叉
        new_population = []
        while len(new_population) < population_size:
            parent1, parent2 = random.sample(selected_population, 2)
            child1, child2 = crossover(parent1, parent2)
            new_population.append(mutate(child1, mutation_rate))
            new_population.append(mutate(child2, mutation_rate))

        # 变异
        population = [mutate(solution, mutation_rate) for solution in new_population]

    # 返回最优解
    best_solution = population[fitness.index(max(fitness))]
    return best_solution

# 调用遗传算法求解最优解
best_solution = genetic_algorithm(cost_matrix, truck_capacity, truck_rent, initial_goods,
    final_goods, population_size=100, mutation_rate=0.1, max_generations=100)
# 打印最优解
print("最优解:", best_solution)

```

附录 8 问题二调用 CIM 模拟器代码.py

```
import kaiwu as kw
import numpy as np

user_id = "用户 ID"
sdk_code = "SDK 授权码"

# 初始化 Kaiwu SDK
kw.license.init(user_id=user_id, sdk_code=sdk_code)

# 从附录 3 中获取 QUBO 矩阵
Q = np.zeros((len(idx) + len(idx_air), len(idx) + len(idx_air)))
# ... (此处省略构建 Q 矩阵的代码) ...

# 调用 Kaiwu SDK 的 CIM 模拟器
output = kw.cim.simulator(
    matrix=Q,
    pump=0.7,
    noise=0.01,
    laps=50,
    dt=0.1,
    normalization=0.3,
    iterations=10
)

# 调用 Kaiwu SDK 的最优解采样器
opt = kw.sampler.optimal_sampler(matrix=Q, output=output, num_samples=0)

# 解析结果
best_solution = opt[0][0]
min_cost = (np.sum(Q) - np.dot(Q, best_solution).dot(best_solution)) / 4

# 输出结果
print(f"最小成本为: {min_cost} 元")
print(f"最优方案: {best_solution}")
```

附录 9 问题二调用模拟退火求解器代码.py

```
import kaiwu as kw
import numpy as np

user_id = "用户 ID"
sdk_code = "SDK 授权码"

# 初始化 Kaiwu SDK
kw.license.init(user_id=user_id, sdk_code=sdk_code)

# 从附录 3 中获取 QUBO 矩阵
Q = np.zeros((len(idx) + len(idx_air), len(idx) + len(idx_air)))
# ... (此处省略构建 Q 矩阵的代码) ...
```

```

# 设置模拟退火求解器参数
initial_temperature = 100
alpha = 0.99
cutoff_temperature = 0.001
iterations_per_t = 10
size_limit = 10

# 创建模拟退火求解器
sa_worker = kw.classical.SimulatedAnnealingOptimizer(
    initial_temperature=initial_temperature,
    alpha=alpha,
    cutoff_temperature=cutoff_temperature,
    iterations_per_t=iterations_per_t,
    size_limit=size_limit
)

# 调用模拟退火求解器
solution = sa_worker.solve(matrix=Q)

# 解析结果
best_solution = solution[0]
min_cost = (np.sum(Q) - np.dot(Q, best_solution).dot(best_solution)) / 4

# 输出结果
print(f"最小成本为: {min_cost} 元")
print(f"最优方案: {best_solution}")

```

附录 10 问题二贪心算法主要代码.py

```

#假定以下变量已从题干或之前代码求出：
# cost_matrix: 存储城市间运输成本的矩阵
# truck_capacity: 存储卡车载货量的列表
# truck_rent: 存储卡车日租金的列表
# initial_goods_company1: 存储公司一每个城市初始货物量的列表
# initial_goods_company2: 存储公司二每个城市初始货物量的列表
# final_goods_company1: 存储公司一每个城市最终货物量的列表
# final_goods_company2: 存储公司二每个城市最终货物量的列表

def greedy_algorithm_collaboration_subproblems(cost_matrix, truck_capacity, truck_rent,
initial_goods_company1, initial_goods_company2, final_goods_company1,
final_goods_company2):
    # 路径选择问题
    solution, total_cost = greedy_algorithm(cost_matrix, truck_capacity, truck_rent,
initial_goods_company1, initial_goods_company2, final_goods_company1,
final_goods_company2)

    # 货物分配问题
    for path in solution:
        i, j, truck_type = path
        # 按照载货量从大到小排序卡车类型
        sorted_truck_types = sorted(range(len(truck_capacity)), key=lambda x:

```

```

truck_capacity[x], reverse=True)
    # 分配货物到卡车上
    for truck_type in sorted_truck_types:
        if truck_capacity[truck_type] <= initial_goods_company1[i] +
initial_goods_company2[i]:
            truck_load = truck_capacity[truck_type]
            initial_goods_company1[i] -= truck_load * 0.5
            initial_goods_company2[i] -= truck_load * 0.5
            final_goods_company1[j] += truck_load * 0.5
            final_goods_company2[j] += truck_load * 0.5
            break

    # 拼货方案问题
    for path in solution:
        i, j, truck_type = path
        # 计算两公司单独运输的成本
        cost_company1 = cost_matrix[i][j] + truck_rent[truck_type] * (cost_matrix[i][j] /
truck_capacity[truck_type])
        cost_company2 = cost_matrix[i][j] + truck_rent[truck_type] * (cost_matrix[i][j] /
truck_capacity[truck_type])
        # 计算拼货运输的成本
        cost_collaboration = cost_matrix[i][j] + truck_rent[truck_type] * (cost_matrix[i][j] /
(truck_capacity[truck_type] * 2))
        # 选择成本更低的方案
        if cost_collaboration < cost_company1 + cost_company2:
            total_cost -= (cost_company1 + cost_company2) - cost_collaboration
            # 更新运输方案
            solution.remove(path)
            solution.append((i, j, truck_type, 2)) # 2 表示拼货运输

    # 返回运输方案和总成本
    return solution, total_cost

# 调用贪心算法求解最优解
solution, total_cost = greedy_algorithm_collaboration_subproblems(cost_matrix, truck_capacity,
truck_rent, initial_goods_company1, initial_goods_company2, final_goods_company1,
final_goods_company2)

# 打印最优解
print("两公司运输方案:", solution)
print("总成本:", total_cost)

```

附录 11 问题二遗传算法主要代码.py

```

import random

# 假设以下变量已经定义：
# cost_matrix: 存储城市间运输成本的矩阵
# truck_capacity: 存储卡车载货量的列表
# truck_rent: 存储卡车日租金的列表
# initial_goods_company1: 存储公司一每个城市初始货物量的列表
# initial_goods_company2: 存储公司二每个城市初始货物量的列表

```

```

# final_goods_company1: 存储公司一每个城市最终货物量的列表
# final_goods_company2: 存储公司二每个城市最终货物量的列表

# 编码函数
def encode_solution(i, j, k, n, p):
    return [i, j, k, n, p]

# 解码函数
def decode_solution(solution):
    i, j, k, n, p = solution
    return i, j, k, n, p

# 适应度函数
def fitness_function(solution):
    i, j, k, n, p = decode_solution(solution)
    cost = cost_matrix[i][j] + truck_rent[k] * (cost_matrix[i][j] / truck_capacity[k])
    cost += cost_matrix[i][j] + truck_rent[n] * (cost_matrix[i][j] / truck_capacity[n])
    if p == 1:
        cost *= 0.5 # 拼货运输成本减半
    return 1 / cost

# 选择函数
def select(population, fitness):
    # 使用轮盘赌选择方法
    total_fitness = sum(fitness)
    cumulative_fitness = [sum(fitness[:i+1]) for i in range(len(fitness))]
    selected_indices = []
    for _ in range(len(population)):
        r = random.uniform(0, total_fitness)
        for i in range(len(cumulative_fitness)):
            if r < cumulative_fitness[i]:
                selected_indices.append(i)
                break
    return [population[i] for i in selected_indices]

# 交叉函数
def crossover(parent1, parent2):
    # 使用单点交叉方法
    r = random.randint(1, len(parent1) - 1)
    child1 = parent1[:r] + parent2[r:]
    child2 = parent2[:r] + parent1[r:]
    return child1, child2

# 变异函数
def mutate(solution, mutation_rate):
    if random.random() < mutation_rate:
        r = random.randint(0, len(solution) - 1)
        solution[r] = random.randint(0, len(solution) - 1)
    return solution

# 遗传算法主函数
def genetic_algorithm(cost_matrix, truck_capacity, truck_rent, initial_goods_company1,
initial_goods_company2, final_goods_company1, final_goods_company2, population_size,

```

```

mutation_rate, max_generations):
    # 初始化种群
    population = [encode_solution(random.randint(0, len(cost_matrix)-1), random.randint(0,
len(cost_matrix[0])-1), random.randint(0, len(truck_capacity)-1), random.randint(0,
len(truck_capacity)-1), random.randint(0, 1)) for _ in range(population_size)]

    for generation in range(max_generations):
        # 计算适应度
        fitness = [fitness_function(solution) for solution in population]

        # 选择
        selected_population = select(population, fitness)

        # 交叉
        new_population = []
        while len(new_population) < population_size:
            parent1, parent2 = random.sample(selected_population, 2)
            child1, child2 = crossover(parent1, parent2)
            new_population.append(mutate(child1, mutation_rate))
            new_population.append(mutate(child2, mutation_rate))

        # 变异
        population = [mutate(solution, mutation_rate) for solution in new_population]

    # 返回最优解
    best_solution = population[fitness.index(max(fitness))]
    return best_solution

# 调用遗传算法求解最优解
best_solution = genetic_algorithm(cost_matrix, truck_capacity, truck_rent,
initial_goods_company1, initial_goods_company2, final_goods_company1,
final_goods_company2, population_size=100, mutation_rate=0.1, max_generations=100)

# 打印最优解
print("最优解:", best_solution)

```